

HANDOUT PEMBELAJARAN — SERI 2 dari 3

Spring MVC

REST API dengan @RestController

@RestController • ResponseEntity • HTTP Status Code • Postman

Tujuan

Membangun REST API yang bisa dikonsumsi aplikasi lain atau frontend

Durasi

1,5 Jam (90 Menit)

Tools

IntelliJ • Postman • Browser

Setelah belajar materi ini, mahasiswa mampu:

1. Memahami konsep REST API dan perbedaannya dengan web biasa (HTML)
2. Membuat @RestController dan endpoint untuk setiap operasi CRUD
3. Memahami anotasi HTTP Method: @GetMapping, @PostMapping, @PutMapping, @DeleteMapping
4. Menggunakan @PathVariable dan @RequestParam untuk menerima data dari URL
5. Menggunakan @RequestBody untuk menerima data JSON dari body request
6. Memahami dan menggunakan HTTP Status Code yang tepat
7. Membungkus response dengan ResponseEntity
8. Menguji semua endpoint menggunakan Postman

Agenda Pembelajaran

Waktu	Durasi	Topik	Metode
0:00 – 0:05	5 menit	Pembukaan: Apa itu REST API?	Presentasi
0:05 – 0:15	10 menit	Konsep REST & @RestController	Presentasi + Diskusi
0:15 – 0:25	10 menit	Setup Postman & project baru	Demo Live
0:25 – 0:45	20 menit	Membuat endpoint GET & POST	Demo + Hands-on
0:45 – 1:00	15 menit	Endpoint PUT, PATCH & DELETE	Demo + Hands-on
1:00 – 1:15	15 menit	HTTP Status Code & ResponseEntity	Presentasi + Demo
1:15 – 1:25	10 menit	Testing lengkap dengan Postman	Hands-on
1:25 – 1:30	5 menit	Tanya jawab & ringkasan	Diskusi

1.1 Apa itu REST API?

REST (Representational State Transfer) adalah gaya arsitektur untuk membangun API yang berkomunikasi lewat HTTP. API (Application Programming Interface) adalah pintu masuk bagi aplikasi lain untuk mengakses data atau fitur dari server kita.

Analogi Sederhana

Bayangkan REST API seperti pelayan di restoran:

- Kamu (client) memesan makanan → Client mengirim HTTP Request
- Pelayan (server/API) membawa pesanan → Server memproses dan kirim Response
- Menu yang tersedia → Endpoint yang terdaftar di API
- Format pesanan (JSON) → Bahasa yang digunakan client & server

1.2 Perbedaan Web Biasa vs REST API

Aspek	Web Biasa (@Controller)	REST API (@RestController)
Yang dikembalikan	Halaman HTML (tampilan)	Data mentah (JSON / XML)
Dikonsumsi oleh	Browser manusia	Aplikasi lain, Mobile App, Frontend JS
Cara testing	Buka di browser	Postman, cURL, atau kode program
Contoh response	Halaman web lengkap	{ "id": 1, "nama": "Budi" }
Format data	HTML	JSON (paling umum) atau XML

1.3 Prinsip REST — CRUD & HTTP Method

REST API menggunakan HTTP Method untuk menentukan jenis operasi yang dilakukan. Setiap operasi CRUD dipetakan ke HTTP Method tertentu:

HTTP Method	Operasi CRUD	Contoh URL	Fungsi
GET	Read	/api/users	Ambil/baca data
POST	Create	/api/users	Buat data baru
PUT	Update	/api/users/1	Update semua field data
DELETE	Delete	/api/users/1	Hapus data

2.1 Dependensi di pom.xml

Untuk REST API, kita hanya butuh dependensi Spring Web (tidak perlu Thymeleaf):

```
<dependencies>
  <!-- Spring Web — mengaktifkan @RestController & HTTP handling -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Lombok — mengurangi getter/setter boilerplate -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

  <!-- DevTools — hot reload -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
</dependencies>
```

2.2 Model Class (Data yang Digunakan API)

```
package com.example.restdemo.model;

import lombok.*;

@Data          // Lombok: generate getter, setter, toString, equals
@NoArgsConstructor // Constructor tanpa argumen
@AllArgsConstructor // Constructor semua argumen
@Builder       // Builder pattern
public class Produk {

    private Long    id;
    private String  nama;
    private String  kategori;
    private Double  harga;
    private Integer stok;
}
```

2.3 Membuat @RestController

@RestController adalah @Controller yang secara otomatis mengubah semua return value menjadi JSON — kita tidak perlu menulis kode konversi sama sekali.

```
package com.example.restdemo.controller;

import com.example.restdemo.model.Produk;
import org.springframework.web.bind.annotation.*;
import java.util.*;

// @RestController = @Controller + @ResponseBody
// Semua method otomatis return JSON
@RestController

// @RequestMapping → semua endpoint di class ini berawalan /api/produk
@RequestMapping("/api/produk")
public class ProdukController {

    // Simulasi database sementara (pakai List di memory)
    // Di materi berikutnya akan diganti dengan database sungguhan
    private List<Produk> daftarProduk = new ArrayList<>(List.of(
        new Produk(1L, "Laptop", "Elektronik", 8500000.0, 10),
        new Produk(2L, "Mouse", "Elektronik", 250000.0, 50),
        new Produk(3L, "Meja Kerja", "Furnitur", 1200000.0, 5)
    ));

    private Long nextId = 4L; // Counter ID berikutnya
}
```

Mengapa Pakai List Sementara?

Saat ini kita belum belajar koneksi database. List di memory cukup untuk memahami cara kerja REST API. Di handout berikutnya (Seri 3), List ini akan diganti database MySQL.

3

Endpoint GET & POST

20 menit

3.1 GET — Mengambil Data

GET

/api/produk

Ambil semua produk

GET

/api/produk/{id}

Ambil produk berdasarkan ID

GET

/api/produk/search?nama=xxx

Cari produk berdasarkan nama

```
// GET /api/produk → kembalikan semua produk
@GetMapping
public List<Produk> getAllProduk() {
    return daftarProduk;
    // Spring otomatis convert List<Produk> → JSON Array
}

// GET /api/produk/1 → produk dengan id = 1
// @PathVariable → ambil nilai {id} dari URL
@GetMapping("/{id}")
public Produk getProdukById(@PathVariable Long id) {
    return daftarProduk.stream()
        .filter(p -> p.getId().equals(id))
        .findFirst()
        .orElseThrow(() -> new RuntimeException("Produk tidak ditemukan: " +
id));
}

// GET /api/produk/search?nama=laptop
// @RequestParam → ambil nilai dari query parameter (?nama=...)
@GetMapping("/search")
public List<Produk> searchProduk(@RequestParam String nama) {
    String keyword = nama.toLowerCase();
    return daftarProduk.stream()
        .filter(p -> p.getNama().toLowerCase().contains(keyword))
        .toList();
}
```

3.2 Contoh Response JSON dari GET

GET /api/produk → 200 OK

```
[
  { "id": 1, "nama": "Laptop",      "kategori": "Elektronik", "harga": 8500000.0,
    "stok": 10 },
  { "id": 2, "nama": "Mouse",      "kategori": "Elektronik", "harga": 250000.0,
    "stok": 50 },
  { "id": 3, "nama": "Meja Kerja", "kategori": "Furnitur",   "harga": 1200000.0,
    "stok": 5  }
]
```

GET /api/produk/1 → 200 OK

```
{
  "id": 1,
  "nama": "Laptop",
  "kategori": "Elektronik",
  "harga": 8500000.0,
  "stok": 10
}
```

3.3 POST — Membuat Data Baru

POST	/api/produk	Tambah produk baru (kirim JSON di body)
-------------	-------------	---

```
// POST /api/produk → tambah produk baru
// @RequestBody → ambil data dari body request (JSON)
// Spring otomatis convert JSON → object Produk
@PostMapping
public Produk tambahProduk(@RequestBody Produk produkBaru) {
    // Set ID otomatis
    produkBaru.setId(nextId++);

    // Simpan ke list
    daftarProduk.add(produkBaru);

    // Kembalikan produk yang baru disimpan
    return produkBaru;
    // HTTP Status default: 200 OK
    // (Kita akan perbaiki ke 201 Created di bagian ResponseEntity)
}
```

3.4 Perbedaan @PathVariable vs @RequestParam vs @RequestBody

Anotasi	Data dari mana?	Contoh URL/Request	Contoh di kode
@PathVariable	Bagian dari path URL	GET /produk/42	@PathVariable Long id
@RequestParam	Query string di URL	GET /produk?nama=laptop	@RequestParam String nama
@RequestBody	Body request (JSON)	POST body: {"nama":"TV"}	@RequestBody Produk p

💡 Kapan Pakai Yang Mana?

@PathVariable → Untuk identifikasi resource spesifik: GET /users/5 (user dengan ID 5)

@RequestParam → Untuk filter/pencarian opsional: GET /users?kota=Jakarta

@RequestBody → Untuk kirim data kompleks (JSON) ke POST/PUT endpoint

4

Endpoint PUT, PATCH & DELETE

15 menit

4.1 PUT — Update Seluruh Data

PUT

/api/produk/{id}

Update semua field produk

```
// PUT /api/produk/1 → update produk dengan id=1
// Catatan: PUT mengganti SELURUH data (semua field harus dikirim)
@PutMapping("/{id}")
public Produk updateProduk(@PathVariable Long id,
                           @RequestBody Produk produkBaru) {
    // Cari index produk di list
    for (int i = 0; i < daftarProduk.size(); i++) {
        if (daftarProduk.get(i).getId().equals(id)) {
            produkBaru.setId(id); // Pertahankan ID yang sama
            daftarProduk.set(i, produkBaru); // Ganti dengan data baru
            return produkBaru;
        }
    }
    throw new RuntimeException("Produk tidak ditemukan: " + id);
}
```

4.2 PATCH — Update Sebagian Data

PATCH

/api/produk/{id}/stok

Update hanya field stok

```
// PATCH /api/produk/1/stok?jumlah=100
// Berbeda dari PUT: PATCH hanya update sebagian field saja
@PatchMapping("/{id}/stok")
public Produk updateStok(@PathVariable Long id,
                        @RequestParam Integer jumlah) {
    Produk produk = daftarProduk.stream()
        .filter(p -> p.getId().equals(id))
        .findFirst()
        .orElseThrow(() -> new RuntimeException("Produk tidak ditemukan: " +
            id));

    produk.setStok(jumlah); // Hanya update field stok
    return produk;
}
```

4.3 DELETE — Menghapus Data

DELETE

/api/produk/{id}

Hapus produk berdasarkan ID

```
// DELETE /api/produk/1 → hapus produk dengan id=1
@DeleteMapping("/{id}")
```

```
public String hapusProduk(@PathVariable Long id) {
    boolean berhasil = daftarProduk
        .removeIf(p -> p.getId().equals(id));

    if (!berhasil) {
        throw new RuntimeException("Produk tidak ditemukan: " + id);
    }

    return "Produk dengan ID " + id + " berhasil dihapus";
    // Nanti kita perbaiki menggunakan ResponseEntity
}
```

4.4 PUT vs PATCH — Kapan Pakai Yang Mana?

	PUT	PATCH
Operasi	Update keseluruhan data	Update sebagian data saja
Field yang dikirim	SEMUA field wajib dikirim	Hanya field yang berubah
Jika field tidak dikirim	Field jadi null / default	Field tidak berubah
Contoh use case	Edit profil lengkap	Ubah hanya password saja

5 HTTP Status Code & ResponseEntity

15 menit

5.1 Apa itu HTTP Status Code?

Setiap HTTP response memiliki status code — angka 3 digit yang menunjukkan hasil dari request. Client (Postman/browser) menggunakan status code ini untuk menentukan apakah request berhasil atau gagal.

Kelompok	Range	Arti Umum
2xx	200 – 299	Request BERHASIL
3xx	300 – 399	Redirect ke URL lain
4xx	400 – 499	Error dari sisi CLIENT (salah request)
5xx	500 – 599	Error dari sisi SERVER (bug/crash)

5.2 Status Code yang Paling Sering Digunakan di REST API

Kode	Nama	Kapan Digunakan
200	OK	GET, PUT, PATCH berhasil
201	Created	POST berhasil — data baru berhasil dibuat
204	No Content	DELETE berhasil — tidak ada data dikembalikan
400	Bad Request	Request tidak valid — misal data wajib tidak diisi
404	Not Found	Data yang dicari tidak ada — misal GET /produk/999
500	Internal Server Error	Error di sisi server — bug, null pointer, dll

5.3 ResponseEntity — Mengontrol Response Sepenuhnya

ResponseEntity adalah wrapper yang memungkinkan kita mengatur HTTP Status Code, header, dan body response secara eksplisit dan tepat.

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

@RestController
@RequestMapping("/api/produk")
public class ProdukController {

    // GET → 200 OK
    @GetMapping
    public ResponseEntity<List<Produk>> getAllProduk() {
        return ResponseEntity.ok(daftarProduk);
        // ResponseEntity.ok() = status 200 + body
    }

    // GET by ID → 200 OK atau 404 Not Found
    @GetMapping("/{id}")
```

```

public ResponseEntity<Produk> getProdukById(@PathVariable Long id) {
    return daftarProduk.stream()
        .filter(p -> p.getId().equals(id))
        .findFirst()
        .map(p -> ResponseEntity.ok(p)) // Kalau ada → 200 OK
        .orElse(ResponseEntity.notFound().build()); // Kalau tidak ada → 404
}

// POST → 201 Created
@PostMapping
public ResponseEntity<Produk> tambahProduk(@RequestBody Produk produkBaru) {
    produkBaru.setId(nextId++);
    daftarProduk.add(produkBaru);

    // HTTP 201 Created — menandakan resource baru berhasil dibuat
    return ResponseEntity
        .status(HttpStatus.CREATED) // ← Set status code 201
        .body(produkBaru); // ← Set body response
}

// PUT → 200 OK atau 404
@PutMapping("/{id}")
public ResponseEntity<Produk> updateProduk(@PathVariable Long id,
                                           @RequestBody Produk data) {
    for (int i = 0; i < daftarProduk.size(); i++) {
        if (daftarProduk.get(i).getId().equals(id)) {
            data.setId(id);
            daftarProduk.set(i, data);
            return ResponseEntity.ok(data);
        }
    }
    return ResponseEntity.notFound().build(); // 404
}

// DELETE → 204 No Content atau 404
@DeleteMapping("/{id}")
public ResponseEntity<Void> hapusProduk(@PathVariable Long id) {
    boolean berhasil = daftarProduk.removeIf(p -> p.getId().equals(id));

    if (berhasil) {
        return ResponseEntity.noContent().build(); // 204 — berhasil, no body
    } else {
        return ResponseEntity.notFound().build(); // 404 — tidak ditemukan
    }
}
}

```

5.4 Cara Cepat Membuat ResponseEntity

Kode	Status	Keterangan
<code>ResponseEntity.ok(body)</code>	200 OK	Berhasil, dengan body
<code>ResponseEntity.ok().build()</code>	200 OK	Berhasil, tanpa body
<code>ResponseEntity.status(HttpStatus.CREATED).body(data)</code>	201 Created	Data baru dibuat
<code>ResponseEntity.noContent().build()</code>	204 No Content	Berhasil, body kosong

ResponseEntity.notFound().build()	404 Not Found	Data tidak ada
ResponseEntity.badRequest().body(msg)	400 Bad Request	Request tidak valid
ResponseEntity.internalServerError().build()	500 Internal Error	Error server

6.1 Mengenal Postman

Postman adalah aplikasi untuk menguji REST API. Kita bisa mengirim berbagai jenis HTTP request (GET, POST, PUT, DELETE) dan melihat response dari server secara detail.

Download Postman

<https://www.postman.com/downloads/> — tersedia untuk Windows, Mac, dan Linux

6.2 Mengenal Tampilan Postman

Area	Fungsi
Method Dropdown	Pilih HTTP Method: GET / POST / PUT / PATCH / DELETE
URL Bar	Ketik URL endpoint yang ingin diuji
Params Tab	Isi @RequestParam (query string ?key=value)
Headers Tab	Atur header — misal Content-Type: application/json
Body Tab	Isi @RequestBody (JSON untuk POST/PUT)
Send Button	Kirim request ke server
Response Body	Lihat JSON yang dikembalikan server
Status Code	Lihat angka status (200, 201, 404, dll)
Time	Waktu yang dibutuhkan server untuk respond

6.3 Panduan Testing Setiap Endpoint

TEST 1 — GET /api/produk (Ambil Semua Produk)

Pengaturan	Nilai
Method	GET
URL	http://localhost:8080/api/produk
Headers	(tidak perlu)
Body	(tidak perlu)
Expected Status	200 OK
Expected Response	JSON Array berisi daftar produk

TEST 2 — GET /api/produk/{id} (Produk by ID)

Pengaturan	Nilai
Method	GET
URL	http://localhost:8080/api/produk/1
Expected Status (ada)	200 OK + JSON produk
Expected Status (tidak ada)	404 Not Found

TEST 3 — POST /api/produk (Tambah Produk Baru)

Pengaturan	Nilai
Method	POST
URL	http://localhost:8080/api/produk
Headers	Content-Type: application/json
Body (raw → JSON)	{ "nama": "Keyboard", "kategori": "Elektronik", "harga": 350000, "stok": 25 }
Expected Status	201 Created
Expected Response	JSON produk baru dengan ID terisi

Jangan Lupa Header!

Saat kirim POST/PUT, wajib set header: Key : Content-Type Value : application/json Tanpa header ini, Spring tidak tahu format data yang dikirim dan akan error 415 Unsupported Media Type.

TEST 4 — PUT /api/produk/{id} (Update Produk)

Pengaturan	Nilai
Method	PUT
URL	http://localhost:8080/api/produk/1
Headers	Content-Type: application/json
Body	{ "nama": "Laptop Gaming", "kategori": "Elektronik", "harga": 15000000, "stok": 3 }
Expected Status	200 OK

TEST 5 — PATCH /api/produk/{id}/stok (Update Stok)

Pengaturan	Nilai
Method	PATCH
URL	http://localhost:8080/api/produk/1/stok?jumlah=100
Headers	(tidak perlu, data via query param)
Body	(tidak perlu)
Expected Status	200 OK

TEST 6 — DELETE /api/produk/{id} (Hapus Produk)

Pengaturan	Nilai
Method	DELETE
URL	http://localhost:8080/api/produk/1
Headers	(tidak perlu)
Body	(tidak perlu)
Expected Status (ada)	204 No Content (body kosong)
Expected Status (tidak ada)	404 Not Found

6.4 Cara Membaca Response di Postman

```
// Cek di bagian bawah Postman:

// 1. Status Code
// Terlihat di kanan atas area response
// Contoh: '200 OK', '201 Created', '404 Not Found'

// 2. Time (waktu respons)
// Contoh: '23 ms' — semakin kecil semakin baik

// 3. Body (tab 'Body' → Pretty → JSON)
// Pastikan format JSON benar dan data sesuai ekspektasi

// 4. Jika error, cek console IntelliJ untuk stack trace
```

Ringkasan & Checklist

✓	Kompetensi
☐	Memahami konsep REST API dan perbedaannya dengan aplikasi web HTML
☐	Membuat @RestController dengan @RequestMapping yang benar
☐	Membuat endpoint GET menggunakan @GetMapping
☐	Membuat endpoint POST menggunakan @PostMapping dan @RequestBody
☐	Membuat endpoint PUT menggunakan @PutMapping
☐	Membuat endpoint DELETE menggunakan @DeleteMapping
☐	Memahami perbedaan @PathVariable, @RequestParam, dan @RequestBody
☐	Memahami HTTP Status Code: 200, 201, 204, 400, 404, 500
☐	Menggunakan ResponseEntity untuk mengontrol status code response
☐	Menguji semua endpoint menggunakan Postman dengan benar

Latihan Mandiri

Buat REST API sederhana untuk manajemen Buku dengan field:
id, judul, pengarang, tahun, harga.

Endpoint yang harus dibuat:

GET /api/buku : Ambil semua buku
 GET /api/buku/{id} : Ambil buku by ID (404 jika tidak ada)
 GET /api/buku/search?judul=xxx : Cari buku berdasarkan judul
 POST /api/buku : Tambah buku baru (return 201 Created)
 PUT /api/buku/{id} : Update buku (return 200 atau 404)
 DELETE /api/buku/{id} : Hapus buku (return 204 atau 404)

Gunakan ResponseEntity untuk semua endpoint, lalu uji semua endpoint dengan Postman!

Referensi	URL
Spring REST Docs	https://spring.io/guides/gs/rest-service
ResponseEntity Docs	https://docs.spring.io/spring-framework/docs
Postman Learning	https://learning.postman.com/docs/getting-started
HTTP Status Codes	https://developer.mozilla.org/en-US/docs/Web/HTTP/Status